Lab – Docker Image Tags

To Begin with Lab

Summary

In this lab, we learn how to create and manage Docker image tags. Docker tags are used to identify different versions of the same Docker image. When we make changes to an application, we can build a new Docker image with a different tag, such as v1, v2, or v3. This helps us manage application versions and run containers using a specific version of an image. We also learn that Docker automatically uses the latest tag when no tag is specified.

Understanding Docker Image Tags

A Docker image tag is a label used to identify a particular version of a Docker image.

When we create a Docker image, we can assign a tag using the -t option with the docker build command.

Why do we use Docker image tags?

- **Version Management:** Maintain different versions of an application.
- **Easy Identification:** Identify which version of an image is being used.
- **Application Updates:** Create new images whenever application code changes.
- **Version Selection:** Run containers using a specific image version.
- **Rollback:** Run an older image version if a newer version has problems.

Example:

Suppose we have a static web application.

Initially, we create an image with version 1:

docker build -t static-app:v1 .

After making changes to the HTML file, we create another image:

docker build -t static-app:v2 .

Now we have two versions of the same application:

Image Name	Tag	Description
static-app	v1	First version of the application
static-app	v2	Updated version of the application

Both images can exist simultaneously.

Lab Steps

Step 1: Open Your Static Web Application

• Open your existing static web application project in VS Code.

• Locate the index.html file.

• Initially, your HTML file might contain:



Step 2: Modify the Application

Now change the application content.

• Change the text from Version 1 to Version 2.

• Save the HTML file.

Step 3: Build a New Docker Image with a Tag

Open the terminal inside your project directory.

Execute:

```
docker build -t staticapp:v2 .
```

```
rrathlinux@LAPTOP-OUNOQCC3:/mnt/d/local/Tem$ docker build -t staticapp:v2 .
[+] Building 3.7s (8/8) FINISHED                                                docker:default
 => [internal] load build definition from Dockerfile                                      0.1s
 => => transferring dockerfile: 84B                                                       0.0s
 => [internal] load metadata for docker.io/library/httpd:latest                           2.7s
 => [auth] library/httpd:pull token for registry-1.docker.io                              0.0s
 => [internal] load .dockerignore                                                         0.1s
 => => transferring context: 2B                                                           0.0s
 => [internal] load build context                                                         0.1s
 => => transferring context: 15.40kB                                                      0.0s
 => CACHED [1/2] FROM docker.io/library/httpd:latest@sha256:f6a86043744215e4be90cf8804375d90022a71  0.1s
 => => resolve docker.io/library/httpd:latest@sha256:f6a86043744215e4be90cf8804375d90022a711d892c9  0.1s
 => [2/2] COPY . /usr/local/apache2/htdocs/                                               0.1s
 => exporting to image                                                                    0.4s
 => => exporting layers                                                                   0.2s
 => => exporting manifest sha256:a8559e3656a42609ef4284dd4d50d456f24a87e9c3eac6f73f4d8bcafd449b16  0.0s
 => => exporting config sha256:af5a1a0099cff57a5ced098874e69c3e50a82d7c6b794898a9f8256b25ce4f6b  0.0s
 => => exporting attestation manifest sha256:bd0c6ddf857eaa8ff0c5fdd96bd7f0a70980ba225bbbcb09fbf7e  0.0s
 => => exporting manifest list sha256:05721249f4ae790250bc66826876caff9eba634fc525f983b604e84c7363  0.0s
 => => naming to docker.io/library/staticapp:v2                                           0.0s
 => => unpacking to docker.io/library/staticapp:v2                                        0.1s
rrathlinux@LAPTOP-OUNOQCC3:/mnt/d/local/Tem$
```

Explanation:

- docker build – Builds a new Docker image.
- -t – Assigns a name and tag to the image.
- static-app – Image name.
- v2 – Version 2 tag.
- . – Specifies the current directory as the build context.

Docker will create a new image with the tag v2.

Step 4: Verify Docker Images

Execute:

```
docker images
```

You should see something similar to:

REPOSITORY TAG IMAGE ID

staticapp latest abc123

staticapp v2 xyz456

```
rrathlinux@LAPTOP-OUNOQCC3:/mnt/d/local/Tem$ docker images
                                                                  i Info →  U In Use
IMAGE              ID             DISK USAGE   CONTENT SIZE   EXTRA
staticapp:latest   31e161167dcd       40.2GB          20GB
staticapp:v2       05721249f4ae        175MB        45.3MB
rrathlinux@LAPTOP-OUNOQCC3:/mnt/d/local/Tem$
```

Explanation:

Now we have two versions of our Docker image:

- static-app:latest – Previous image.
- static-app:v2 – Newly created image.

Important: When you build an image without specifying a tag, Docker automatically assigns the latest tag.

Step 5: Check Existing Containers

Execute:

```
docker ps
```

• Check whether any previous containers are available.

```
rrathlinux@LAPTOP-OUNOQCC3:/mnt/d/local/Tem$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED              STATUS              PORTS
MES
a37f9632cd51   staticapp:v2   "httpd-foreground"   About a minute ago   Up About a minute   127.0.0.1:8080->80/tcp, [::1]:8080->80/tcp
aticapp
rrathlinux@LAPTOP-OUNOQCC3:/mnt/d/local/Tem$
```

• Stop and remove the previous container if necessary.

```
docker stop staticapp
docker rm staticapp
```

```
rrathlinux@LAPTOP-OUNOQCC3:/mnt/d/local/Tem$ docker stop a37f9632cd51
a37f9632cd51
rrathlinux@LAPTOP-OUNOQCC3:/mnt/d/local/Tem$ docker rm a37f9632cd51
a37f9632cd51
rrathlinux@LAPTOP-OUNOQCC3:/mnt/d/local/Tem$
```

Step 6: Run a Container Without Specifying a Tag

Now execute:

```
docker run -d -p 8080:80 --name staticapp-v1 staticapp
```

```
rrathlinux@LAPTOP-OUNOQCC3:/mnt/d/local/Tem$ docker run -d -p 8080:80 --name staticapp staticapp:1
6f199183a5236be09d82edc1d052174bd83905cb02cf543774e8d560a7a455ff
rrathlinux@LAPTOP-OUNOQCC3:/mnt/d/local/Tem$
```

What happens here?

Since we haven't specified a tag, Docker automatically uses:

staticapp:latest

This means the container runs using the image tagged latest, not v2.

Open your browser:

http://localhost:8080

You should see the previous version of your application.

Step 7: Stop and Remove the Previous Container

Stop the running container:

```
docker stop staticapp-1
```

```
rrathlinux@LAPTOP-OUNOQCC3:/mnt/d/local/Tem$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED         STATUS         PORTS                                       NAMES
6f199183a523   staticapp:1    "httpd-foreground"   2 minutes ago   Up 2 minutes   127.0.0.1:8080->80/tcp, [::1]:8080->80/tcp   staticapp
rrathlinux@LAPTOP-OUNOQCC3:/mnt/d/local/Tem$ docker stop 6f199183a523
6f199183a523
rrathlinux@LAPTOP-OUNOQCC3:/mnt/d/local/Tem$
```

Remove the container:

```
docker rm staticapp-1
```

```
rrathlinux@LAPTOP-OUNOQCC3:/mnt/d/local/Tem$ docker stop 6f199183a523
6f199183a523
rrathlinux@LAPTOP-OUNOQCC3:/mnt/d/local/Tem$ docker rm 6f199183a523
6f199183a523
rrathlinux@LAPTOP-OUNOQCC3:/mnt/d/local/Tem$
```

Step 8: Run a Container Using the New Image Version

Now specify the image tag explicitly.

Execute:

```
docker run -d -p 8080:80 --name static-app-v2 static-app:v2
```

```
rrathlinux@LAPTOP-OUNOQCC3:/mnt/d/local/Tem$ docker rm 6f199183a523
6f199183a523
rrathlinux@LAPTOP-OUNOQCC3:/mnt/d/local/Tem$ docker run -d -p 8080:80 --name staticapp-v2 staticapp:v2
8722c324d998a85fb2e69efb957c42d37a60fec46dc7be0ec52290c0cd0ba782
rrathlinux@LAPTOP-OUNOQCC3:/mnt/d/local/Tem$
```

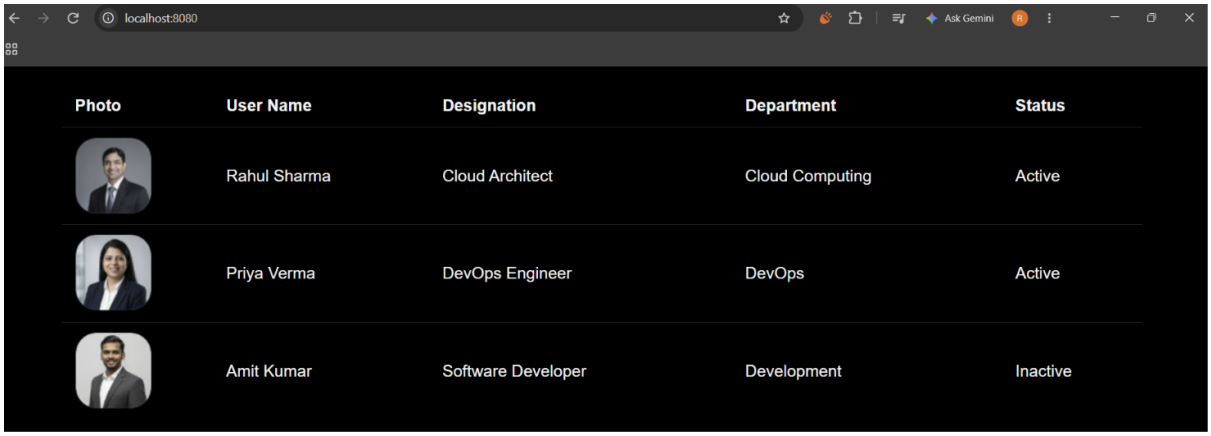
Explanation:

- docker run – Creates and starts a container.
- -d – Runs in detached mode.
- -p 8080:80 – Maps host port 8080 to container port 80.
- --name – Assigns a custom container name.
- static-app:v2 – Uses the version 2 Docker image.

Step 9: Verify the New Application Version

Open your browser:

http://localhost:8080



Now you should see:

Welcome to Static App

Version 2

Your updated application is running successfully.